

CPU & GPU Implementation of the Number Theoretic Transform

Aymane El Jerari
High Performance Computing
Spring 2026
[\[Github Link\]](#)

1. Introducing The Number Theoretic Transform

Multiplying two polynomials of degree n using standard multiplication algorithms requires an execution time proportional to $O(n^2)$. This is because every coefficient of the first polynomial must be multiplied by every coefficient of the second polynomial. For polynomials with low degrees, this computational cost is manageable. However, modern cryptographic protocols (Post Quantum Cryptography algorithms or Fully Homomorphic Encryption schemes) require polynomials with high degrees, typically ranging from $n = 2^{14}$ to $n = 2^{18}$. At this scale, the $O(n^2)$ time complexity grows exponentially making PQC and FHE schemes not practical.

The Number Theoretic Transform (NTT) provides an algorithmic method to reduce this complexity. The NTT operates as the finite-field equivalent of the Fast Fourier Transform (FFT). While the standard FFT operates on complex numbers using floating-point arithmetic, the NTT operates strictly on integers modulo a specific prime number.

The mechanism of the NTT relies on converting the representation of the polynomials. It maps polynomial coefficients from standard “coefficient representation” into an “evaluation form”. This is somewhat analogous to the FFT’s time and frequency domains. In the evaluation domain, the multiplication of two polynomials is achieved by an elementwise multiplication of polynomial roots. This element-wise multiplication reduces the time complexity of the multiplication step to $O(n)$. After this element-wise operation is completed, an Inverse Number Theoretic Transform (iNTT) is applied to map the resulting evaluation form data into the standard polynomial in the coefficient domain. The process of the forward NTT and the iNTT each require $O(n \log n)$ operations. Consequently, the total time complexity of polynomial multiplication is reduced from $O(n^2)$ to $O(n \log n)$. For a polynomial of degree $n = 2^{14}$, this reduces the required operations from over 268 million to approximately 230,000, a reduction of more than 1000x.

During FHE workloads, thousands of sequential polynomial multiplications are required to process the data. Because of this, the NTT is consistently the primary computational bottleneck, accounting for 70% to 90% of the total execution time in standardized FHE schemes. This makes optimizing the NTT one of the main ways of improving the throughput of FHE systems.

2. Background

Fully Homomorphic Encryption

FHE enables secure and fully private computation without requiring trust between the client and the server. It allows a client to encrypt data, send the ciphertexts to an untrusted server, such that processing can be done without every decrypting the ciphertext on the server side. Plaintext data—such as boolean

values, integers, or real numbers—are encoded into polynomial structures. During encryption, these polynomials are modified with cryptographic noise to secure the data against decryption attempts.

Residue Number System

The coefficients of polynomials in FHE schemes require large bit-widths to maintain security and accommodate the growth caused by multiplication. A single coefficient can require up to if not more than 1500 bits. Traditional CPU and GPUs datapaths usually operate on 64-bit hardware registers. Processing an 1500 bit integer requires multi-precision software emulation, which is highly inefficient.

To resolve this hardware constraint, modern FHE implementations use the Residue Number System (RNS). RNS is based on the Chinese Remainder Theorem (CRT). It allows a single large integer to be represented as a sequence of smaller integers, which are the remainders of the large integer divided by a set of mutually coprime moduli. A 1500 bit coefficient can be split into approximately 24 independent 64-bit coefficients. This division creates independent processing channels. Since no data needs to be shared between these RNS channels during addition or multiplication, the operations are parallelizable across available CPU cores or GPU threads. This project leverages limb-wise parallelism to accelerate the NTT.

Data Type and Overflow Considerations

When implementing RNS channels on 64-bit hardware, data type management is necessary to prevent integer overflow. When a processor multiplies two 64-bit integers, the resulting product requires 128 bits of storage space.

To manage this within standard hardware limits, implementations often restrict the RNS base moduli to less than the full data type width. For instance, this project uses `uint32_t` for the RNS coefficients, however, the actual bitwidth of each coefficient is set to 28 better manage overflow. Using 28 bit integers within the 32 bit datatype leaves a computational buffer. Multiple additions can occur consecutively without exceeding the 32-bit boundary. When a multiplication operation is executed, the two 28 bit integers are cast to the 64 bit datatype. The resulting product is 56 bits, which fits safely within the hardware's standard 64-bit register. This data type strategy prevents overflow and avoids the software overhead associated with 128-bit arithmetic, better utilizing the processor's native arithmetic logic units (ALUs).

Modular Arithmetic

The NTT operates within a finite field, meaning that the results of all addition and multiplication operations must be constrained by a modulus q . This requires a modular reduction operation (finding the remainder of division by q) after every arithmetic step. Standard integer division instructions on modern processors are computationally slow, often requiring 15 to 40 clock cycles to complete a single operation compared to 1 to 3 cycles for addition or multiplication.

FHE implementations bypass expensive hardware division by using Barrett reduction. Barrett reduction is based on the principle that dividing a value by q is equivalent to multiplying that value by the inverse of q ($\frac{1}{q}$). Because q is known in advance, implementations precompute a scaled, integer representation of $\frac{1}{q}$. This allows the reduction process to replace the slow hardware division step with a sequence of standard integer operations like multiplication, subtraction and bit shifts.

Barrett Reduction

Input: Value x , modulus q , precomputed constant μ (where $\mu = \text{floor}(2^k / q)$ for word size k)

Output: $x \bmod q$

1. $q1 = \text{floor}((x * \mu) / 2^k)$
2. $r = x - q1 * q$
3. **if** $r \geq q$ **then**
 1. $r = r - q$
4. **return** r

NTT High level characterization

The execution profile of the NTT is characterized by specific compute and memory access patterns. The main operation of the NTT is the butterfly. This consists of a multiply-accumulate operation immediately followed by a modular reduction. This computation puts a lot of pressure on the integer ALUs. Additionally, the NTT requires data to be accessed in a strided pattern. Standard memory systems operate using cache lines, loading adjacent data into the on chip cache blocks. Because of NTT's strided accesses at each stage of the algorithm, it pulls unneeded data into the cache, leading to high cache miss rates. As the polynomial size increases, this memory access pattern shifts the overall bottleneck to become limited by the memory bandwidth.

3. CPU and GPU Implementations of the NTT

Butterfly Network Optimizations

The NTT processes data through a structured routing mechanism known as a butterfly network. This network dictates which memory indices are paired together at each stage of the $O(\log n)$ NTT algorithm. Naive implementations of the NTT required the input array to be sorted into "bit-reversed" order before processing can occur. This reordering is very taxing on the memory system as reordering large polynomials has very low arithmetic intensity. Optimized implementations avoid this by using the Gentleman-Sande (GS) butterfly structure for the forward NTT and the Cooley-Tukey (CT) for the iNTT. These specific butterfly variations remove the previous bit reversal requirement by taking it into account directly within the operational NTT stages. The GS butterfly takes as input the natural polynomial order and outputs a bit reversed order. The CT butterfly takes a bit reversed input and outputs the original polynomial.

CPU Naive Implementation

The baseline implementation is a direct translation of the standard mathematical definition of polynomial multiplication. It executes in $O(n^2)$ time. The implementation relies on a double-nested loop, where the outer loop iterates through the coefficients of the first polynomial, and the inner loop iterates through the coefficients of the second polynomial. Every multiplication operation within the inner loop is immediately followed by a modular reduction step. This implementation is straightforward and is a critical component used to verify correctness of the other implementations.

CPU Fast Implementation

To attain the optimal $O(n \log n)$ complexity, the “CPU Fast” implementation employs the standard NTT architecture but computes the twiddle factors dynamically. Twiddle factors are the algorithmic roots of unity required for multiplication at each node of the butterfly network. By calculating these values on the fly during the algorithm’s execution, the implementation minimizes its memory footprint as it does not require an additional array for storing twiddles. However, computing modular exponentiation on fly is computationally expensive. This design choice shifts the execution bottleneck away from memory bandwidth to compute throughput.

CPU Fast - GS NTT (On-the-fly Twiddles)

Input: Array **a** of size **N**, modulus **q**, primitive root **root**

Output: Transformed array **a** (in-place)

1. **for** len from N down to 2, halving len in each step **do**:
 1. $wlen = (\text{root}^{(N / \text{len})}) \bmod q$
 2. **for** i from 0 to N - 1 step len **do**:
 1. $w = 1$
 2. **for** j from 0 to len / 2 - 1 **do**:
 1. $u = a[i + j]$
 2. $v = a[i + j + \text{len} / 2]$
 3. $a[i + j] = (u + v) \bmod q$
 4. $\text{diff} = (u - v + q) \bmod q$
 5. $a[i + j + \text{len} / 2] = (\text{diff} * w) \bmod q$
 6. $w = (w * wlen) \bmod q$
2. **return** a

CPU Production Implementation

The “CPU Production” implementation is more similar to how production cryptographic libraries implement the algorithm. It executes in $O(n \log n)$ complexity but precomputes all required twiddle factors before the NTT is launched. These computations are performed once during the cryptographic key-generation phase. During the execution of the NTT, the CPU fetches the required twiddle factor directly from memory. While this removes the overhead of modular exponentiation from the inner loop, it doubles the data that must be fetched from memory, increasing the algorithm’s reliance on the on-chip cache.

CPU Production - CT INTT (Precomputed Twiddles)

Input: Array **a** of size **N**, modulus **q**, precomputed inverse twiddles array **inv_omega_pow**, inverse of N **inv_N**

Output: Inverse transformed array **a** (in-place)

1. **for** len from 2 up to N, doubling len in each step **do**:
 1. $\text{step} = N / \text{len}$

```

2. half_len = len / 2
3. for i from 0 to N - 1 step len do:
    1. for j from 0 to half_len - 1 do:
        1. w = inv_omega_pow[j * step]
        2. u = a[i + j]
        3. v = (a[i + j + half_len] * w) mod q
        4. a[i + j] = (u + v) mod q
        5. a[i + j + half_len] = (u - v + q) mod q
    2. for i from 0 to N - 1 do:
        1. a[i] = (a[i] * inv_N) mod q
3. return a

```

GPU Radix-2 vs. Radix-4 Implementations

Running NTT on a GPU is a natural next step as the algorithm contains enough parallelism to leverage the SIMT model. Radix 2 is the baseline GPU implementation. It pairs two coefficient elements per execution thread. Due to the changing stride at each NTT stage, the distance between the paired elements causes uncoalesced accesses to the GPU's global memory. Furthermore, the algorithm requires a global thread synchronization barrier after the completion of every single stage before processing the next which adds additional synchronization latency.

GPU Radix-2 DIF NTT per Stage

Input: Array **a**, precomputed **twiddles**, size **N**, modulus **q**, stage length **len**

Output: Array **a** updated for the current stage

```

1. tid = Global Thread ID (blockIdx.x * blockDim.x + threadIdx.x)
2. if tid >= N / 2 then return
    1. half_len = len / 2
    2. group = tid / half_len (Integer division)
    3. j = tid mod half_len
    4. i = group * len
    5. step = N / len
    6. w = twiddles[j * step]
    7. u = a[i + j]
    8. v = a[i + j + half_len]
    9. a[i + j] = (u + v) mod q
    10. diff = (u - v + q) mod q
    11. a[i + j + half_len] = (diff * w) mod q

```

Radix 4 is an optimized approach that assigns four elements to each thread and calculates a local butterfly network. By processing four elements simultaneously, the total number of required algorithmic stages is reduced by half. In this configuration, the kernel's arithmetic intensity increases as more operations are performed per byte of fetched data. This significantly improves performance as the GPU can more easily

hide memory access latency. The GPU Radix 2 and 4 implementations perform computation by launching the same kernel at each stage of the NTT while adjusting the stride to match the current stage.

GPU Radix-4 DIF NTT per Stage

Input: Array **a**, precomputed **twiddles**, size **N**, modulus **q**, stage length **len**, radix-4 multiplier **W_4**

Output: Array **a** updated for the current radix-4 stage

1. `tid = Global Thread ID (blockIdx.x * blockDim.x + threadIdx.x)`
2. **if** `tid >= N / 4` **then return**
 1. `quarter_len = len / 4`
 2. `group = tid / quarter_len` (Integer division)
 3. `j = tid mod quarter_len`
 4. `i = group * len`
 5. `step = N / len`
 6. `W1_idx = j * step`
 7. `w1 = twiddles[W1_idx]`
 8. `w2 = twiddles[W1_idx * 2]`
 9. `w3 = twiddles[W1_idx * 3]`
10. `u0 = a[i + j]`
11. `u1 = a[i + j + quarter_len]`
12. `u2 = a[i + j + 2 * quarter_len]`
13. `u3 = a[i + j + 3 * quarter_len]`
14. `A = (u0 + u2) mod q`
15. `B = (u0 - u2 + q) mod q`
16. `C = (u1 + u3) mod q`
17. `D = (u1 - u3 + q) mod q`
18. `D = (D * W_4) mod q`
19. `out0 = (A + C) mod q`
20. `out1 = (B + D) mod q`
21. `out2 = (A - C + q) mod q`
22. `out3 = (B - D + q) mod q`
23. `a[i + j] = out0`
24. `a[i + j + quarter_len] = (out1 * w1) mod q`
25. `a[i + j + 2 * quarter_len] = (out2 * w2) mod q`
26. `a[i + j + 3 * quarter_len] = (out3 * w3) mod q`

OpenFHE Integration

For baseline performance comparisons, the custom implementations are evaluated against OpenFHE, a prominent open-source FHE library. OpenFHE can be compiled to use OpenMP to achieve better performance by partitioning RNS limbs across multiple CPU cores. This provides a industry standard multi-threaded $O(n \log n)$ baseline to compare against.

4. Evaluation Results

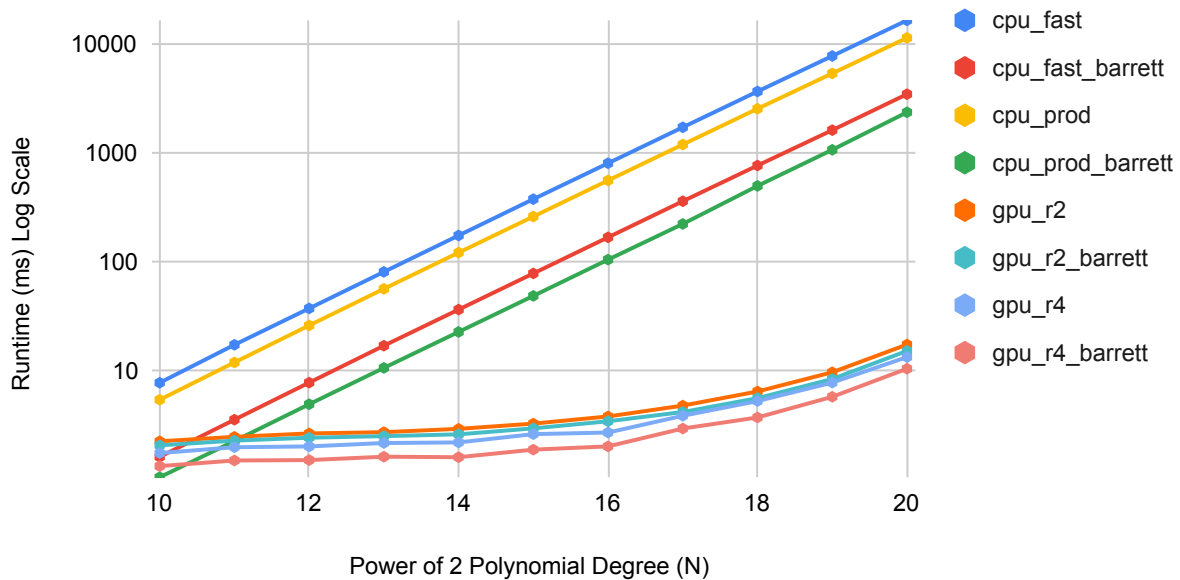
The operational efficiency of the CPU and GPU implementations is assessed utilizing two primary evaluation workflows. The objective of these workflows is to capture end-to-end execution time and low-level hardware utilization metrics.

The results were obtained by running all NTT implementations on the the same system with an AMD EPYC 7H12 CPU and an Nvidia V100 GPU.

Runtime Analysis

Single treaded CPU vs GPU performance

NTT Runtime vs. Polynomial Degree (N = 10 to 20): CPU and GPU, With and Without Barrett Reduction



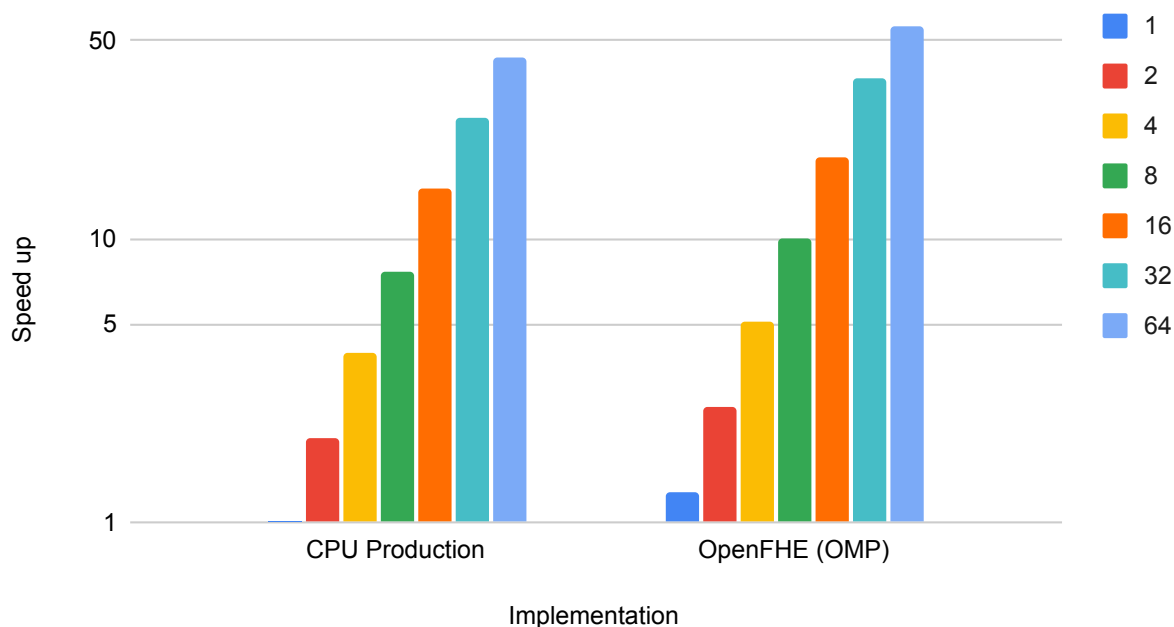
As illustrated in the runtime sweep across polynomial degrees from $N = 10$ to $N = 20$, a clear difference in scaling behavior exists between the CPU and GPU implementations. Across all hardware platforms and algorithmic configurations, using Barrett reduction consistently yields lower execution times compared to the standard modulo reduction. This improvement highlights the efficiency of replacing high-latency integer division instructions with faster bitwise shifts and multiplications, regardless of the underlying processor architecture. Within the single-threaded CPU execution domain, the `cpu_prod` approach maintains a strict performance advantage over the `cpu_fast` computations, proving that precomputing twiddle factors effectively trades a larger memory footprint for an increase in ALU throughput.

When evaluating the parallel device execution, the GPU implementations demonstrate an initial plateau in runtime at lower polynomial degrees. For smaller polynomials, the actual computational workload is negligible, and the end-to-end runtime is heavily dominated by the fixed latencies of PCIe data transfers and the overhead of iterative kernel launches. As a result, the performance differences between the radix 2 and radix 4 architectures remain minimal. However, as the workload expands toward $N = 20$, the massive

SIMD parallelism of the GPU becomes more saturated, allowing the GPU configurations to outperform the CPU baselines. The `gpu_r4_barrett` implementation achieves the lowest overall runtime. This configuration successfully leverages its higher arithmetic intensity to minimize global memory accesses and maximize L2 cache utilization.

CPU Strong Scaling Results

Strong Scaling CPU Prod vs OpenFHE (N=18, L=64)



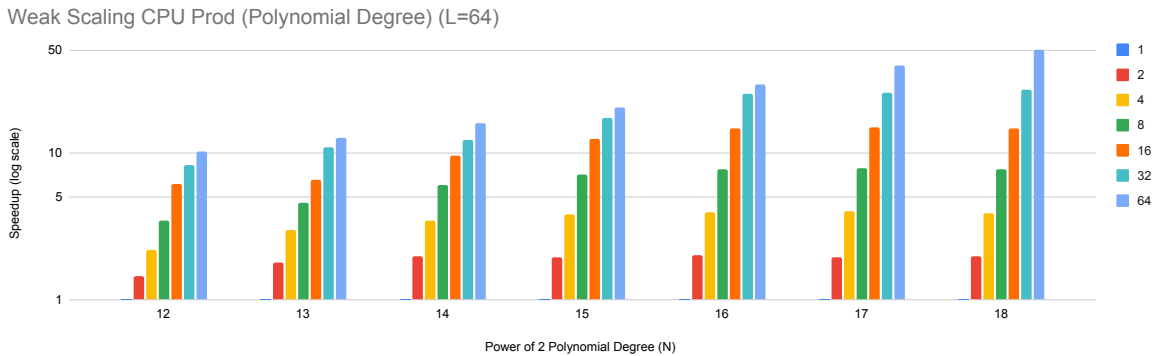
This figure shows the strong scaling results comparing the CPU Production implementation against the multi-threaded OpenFHE implementation for a polynomial degree of $N = 18$ and number of limbs $L = 64$. The evaluation sweeps across hardware thread counts ranging from 1 to 64.

Both implementations achieve almost linear scaling characteristics up to 4 threads, indicating high parallel efficiency and minimal synchronization overhead. The CPU Production code tracks tightly with the theoretical ideal speedup (e.g., achieving a 15.06x speedup on 16 threads). The OpenFHE implementation is consistently achieving higher performance, starting with a 1.28x speedup at 1 thread relative to the unthreaded baseline, and maintaining this advantage across the scaling curve.

As the thread count increases to 32 and 64, we start seeing diminishing returns as the speedup curves for both implementations begin to deviate from the ideal trajectory. At 64 threads, the CPU Production implementation achieves a 43.3x speedup, while OpenFHE achieves a 56.11x speedup. This performance degradation at higher thread counts is a sign that the workload reaching the limits of the underlying CPU architecture, likely due to a combination of memory bandwidth saturation and cache contention among the numerous cores. Despite the sub-linear scaling at higher thread counts, the custom CPU implementation successfully mirrors the scaling behavior of the better optimized OpenFHE framework.

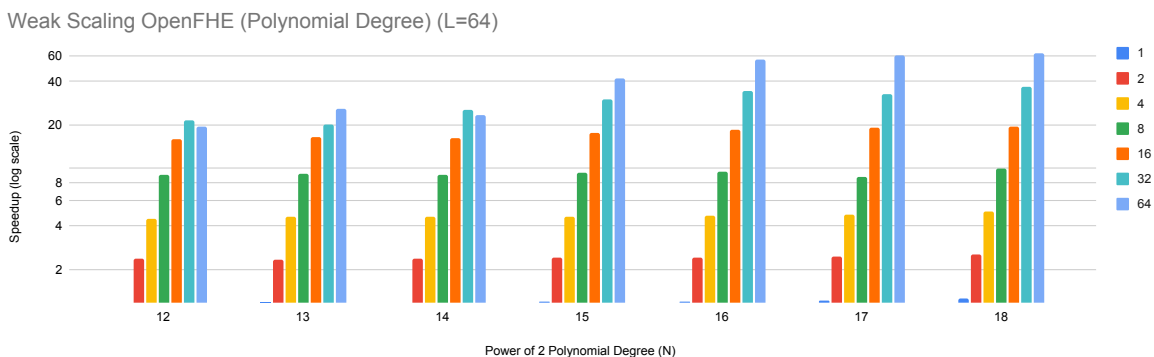
CPU Weak Scaling Results (Scaling Polynomial Degree)

CPU Production



This figure plots the weak scaling behavior of the CPU Production implementation across polynomial degrees ranging from $N = 12$ to $N = 18$, utilizing up to 64 hardware threads. At lower polynomial degrees ($N = 12$ to 14), the implementation struggles to achieve efficient parallel scaling. Even when allocating 64 threads, the maximum observed speedup plateaus between 10x and 15x. This indicates that for small input sizes, the computational workload is insufficient to overcome the overhead associated with OpenMP thread creation, context switching, and barrier synchronization. As the polynomial degree increases to $N = 18$, the arithmetic intensity of the transform grows exponentially, providing enough parallelizable work to keep the threads saturated. Consequently, the scaling efficiency improves significantly, achieving a peak speedup of approximately 50.1x on 64 threads, demonstrating that the custom production implementation is heavily stalled by the lack of sufficient workload to mask its parallel overhead.

OpenFHE



The OpenFHE weak scaling analysis demonstrates a more efficient parallelism at scale. As the polynomial degree increases, the framework exhibits near linear scaling, achieving a 62.67x speedup on 64 threads and $N = 18$. However, at smaller polynomial degrees $N = 12$ and $N = 14$ we see a scaling degradation. For these smaller workload sizes, launching 64 threads results in actual performance degradation compared to utilizing 32 threads (e.g. dropping from 21.4x to 19.3x at $N=12$). This performance inversion highlights that the OpenFHE implementation relies on complex internal synchronization mechanisms that heavily penalize the execution time when the computational payload per thread becomes too small.

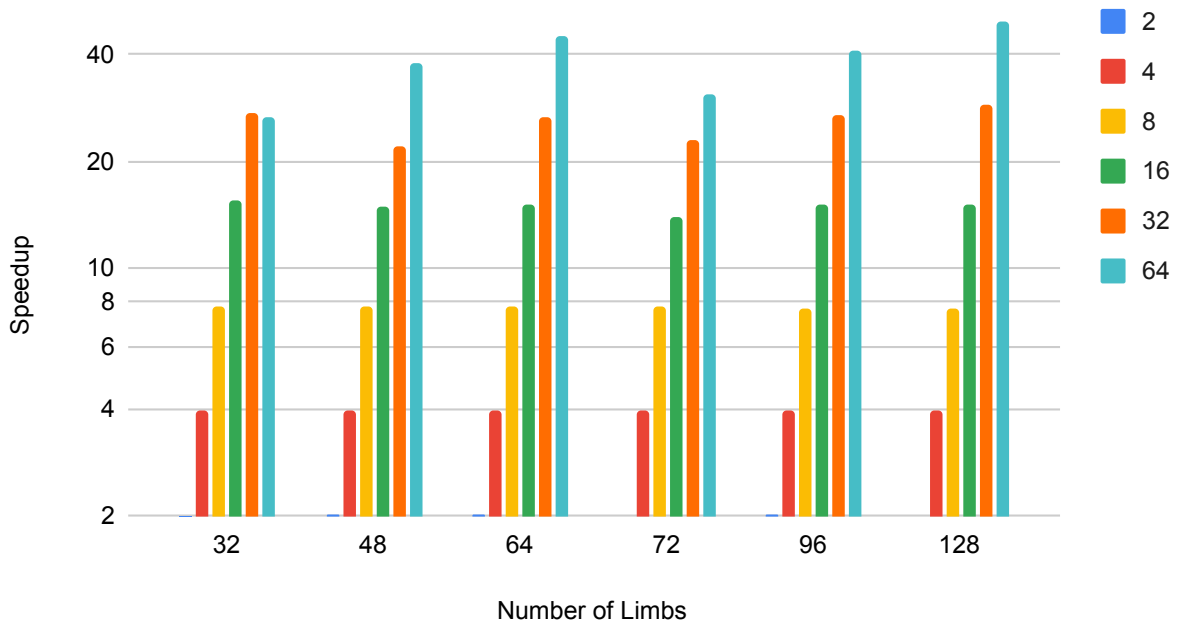
Interestingly, this performance degradation at 64 threads seems to only affect $N = 12$ and $N = 14$ but not the intermediary $N = 13$.

The OpenFHE baseline achieves better scaling capabilities for large workloads. It outperforms the custom CPU Production implementation by a wide margin at $N = 18$ (62.67x versus 50.12x). While the custom CPU Production implementation fails to reach the same peak efficiency at scale, it exhibits more resilient behavior at the lower bounds. Where OpenFHE suffers performance regression at 64 threads for $N = 12$ to 14, the custom implementation plateaus. This difference implies that the custom CPU Production code employs a lower-overhead synchronization model that suffers less thread contention on small data sets.

CPU Weak Scaling Results (Scaling Number of Limbs)

CPU Production

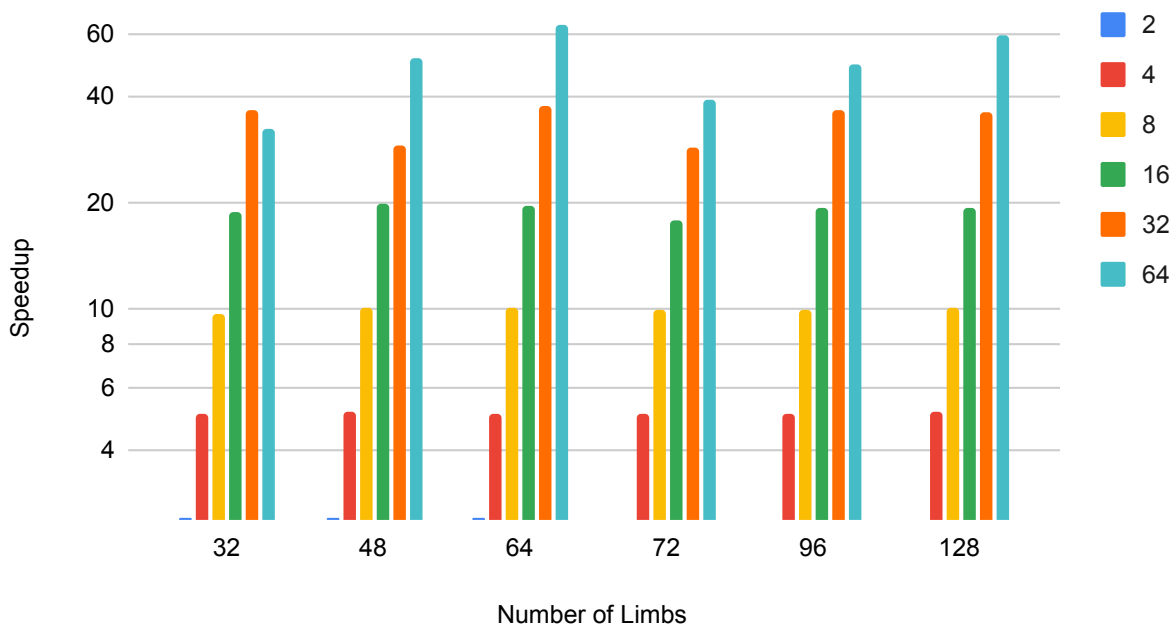
Weak Scaling CPU Prod (# Limbs) (N=18)



This figure plots weak scaling behavior of the custom CPU Production implementation as a function of the number of limbs L , sweeping the number of limbs (L) from 32 to 128 while keeping the polynomial degree fixed at $N = 18$. Similar to the scaling behavior observed with varying polynomial degrees, the implementation exhibits a performance regression at the lowest workload boundary. When $L = 32$, launching 64 threads results in a lower speedup (26.67x) compared to utilizing 32 threads (27.52x). However, as the number of limbs increases, the arithmetic intensity per thread grows. By $L = 128$, the workload is large enough to mask these parallel overheads, allowing the implementation to achieve a much higher peak speedup of approximately 49.67x on 64 threads.

OpenFHE

Weak Scaling OpenFHE (# Limbs) (N=18)



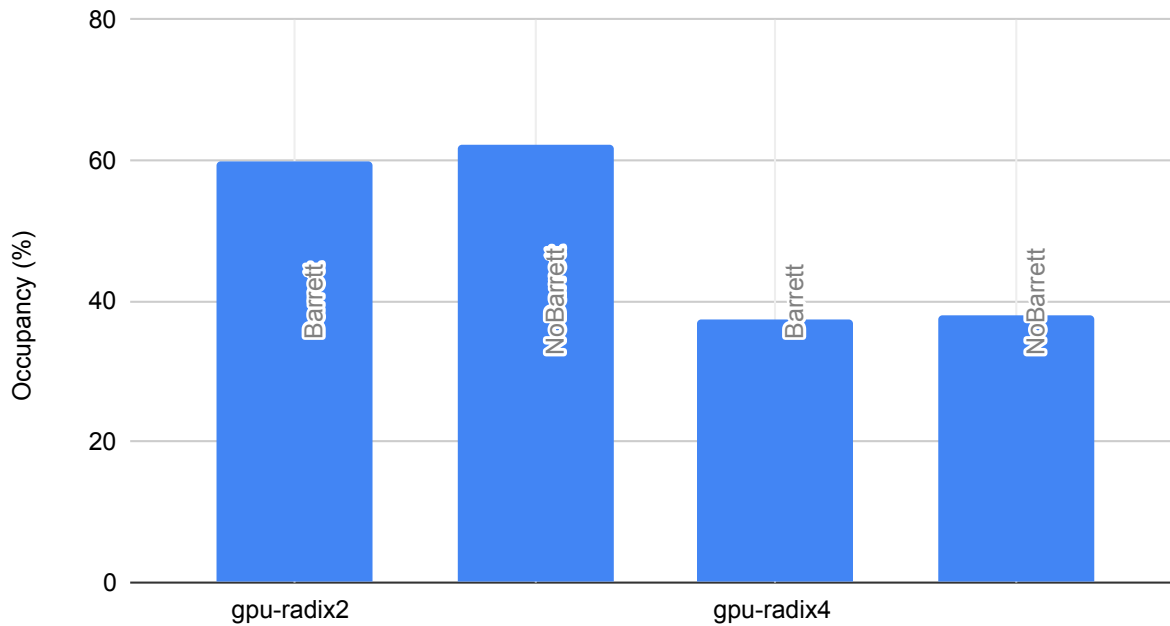
As expected, the OpenFHE implementation is more efficient but sees the same performance degradation at $L = 32$, where moving from 32 to 64 threads causes the speedup to drop from 36.35x to 32.28x. As the workload increases, it manages to achieve almost perfect linear scaling at $L = 64$ with a 63.54x speedup on 64 threads. Interestingly, as the limb count scales further to 96 and 128, the maximum achievable speedup slightly degrades, settling at roughly 59.64x for $L = 128$. This subtle drop in peak parallel efficiency at the highest limb counts suggests that the sheer volume of data being processed per thread is beginning to saturate the underlying hardware's memory bandwidth or maybe exceeds last level cache.

OpenFHE demonstrates a significantly higher peak parallel efficiency, reaching a 63.5x speedup compared to the custom implementation's peak of 49.6x. Both implementations see considerable performance drops at $L = 72$ and $L = 96$ likely due to thread imbalance caused by the "non power of 2" number of limbs.

Nsight Compute Hardware Counter Analysis

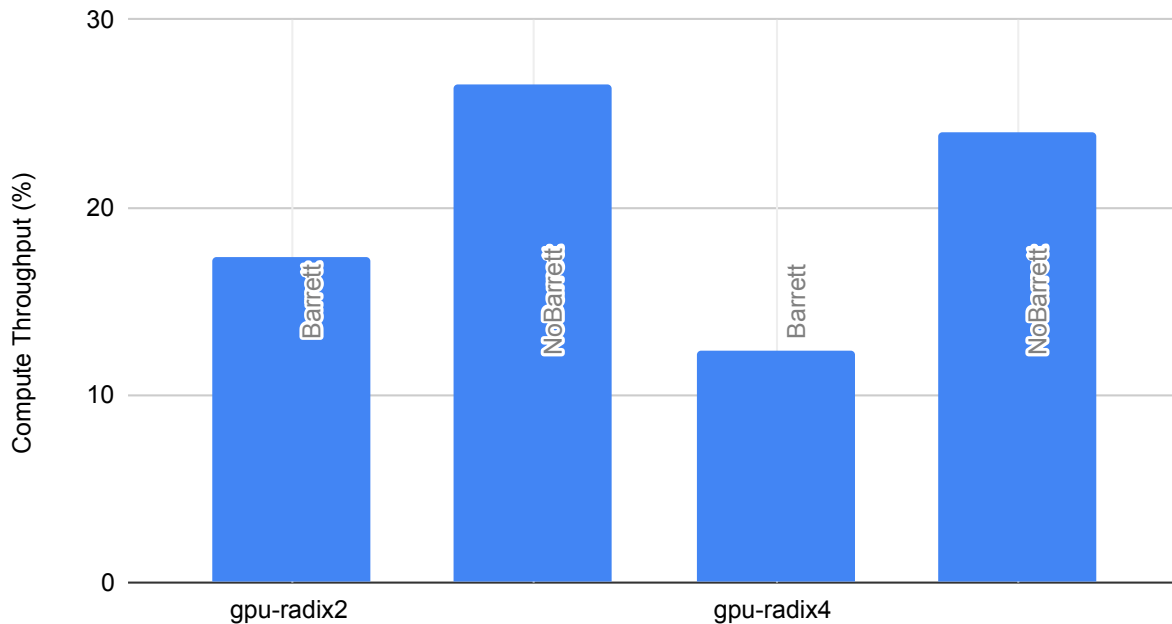
NVIDIA Nsight Compute is used to gather hardware performance counter data on the GPU to compare Radix-2 and Radix-4 implementations as well as the impact of Barrett reduction. Since the GPU implementation architecture relies on a single computational kernel that must be launched iteratively, utilizing different a stride argument at each stage of the forward NTT and iNTT. Due to this, the output generated by Nsight Compute yields disaggregated performance data isolated to each individual kernel launch. To generate end-to-end performance metrics, a Python script is used. This script parses the Nsight Compute .csv files and aggregates the individual kernel launch to give an overview of performance metrics for the complete execution of the full NTT and the full iNTT.

Achieved Hardware Occupancy GPU radix 2 vs radix 4



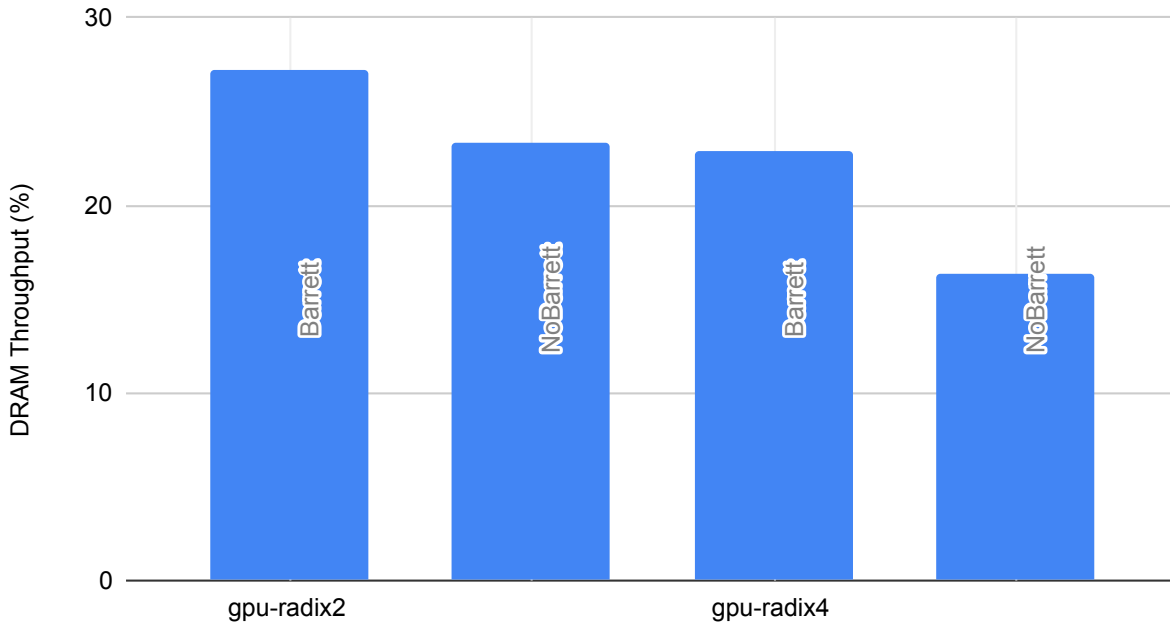
As shown in the figure above, the hardware occupancy of the GPU kernels varies more between radix implementations than between Barrett and NoBarrett implementations. These results are due to the register pressure caused by these different implementations. Radix 4 performs computations on 4 polynomial coefficients and requires more temporary variables before a full stage kernel is completed. Because of the increased register requirement, the maximum number of warps the GPU can manage is reduced. Radix 2 achieves occupancy of about 60% whereas Radix 4 hovers around 38%. Additionally, the Barrett implementation sees a consistent 2-5% reduction in occupancy as additional registers must be allocated such as the precomputed μ factor.

Compute Throughput GPU radix 2 vs radix 4



Across both radix 2 and 4, the NoBarrett implementation utilizes more of the ALU resources performing the expensive modulo reduction operation. Instead, when Barrett reduction is enabled, the compute throughput drops significantly by 10% and 12% for radix 2 and 4 respectively. By optimizing the math with Barrett reduction, the kernel finishes its compute tasks faster than the memory subsystem can supply data. The kernel becomes memory-bound. The SMs spend more time stalled waiting for memory round-trips, which causes the compute throughput percentage to drop. Although radix 4 consistently outperforms radix 2 in end to end runtime, this does not necessarily translate as better usage of the GPU SIMD units. In addition to the increased register requirement, radix 4 also stresses the memory system a lot more, while having lower occupancy to hide the memory fetch latency. Because fewer warps are available for computation while others wait for memory, the compute units sit idle more often, resulting in a lower compute throughput percentage.

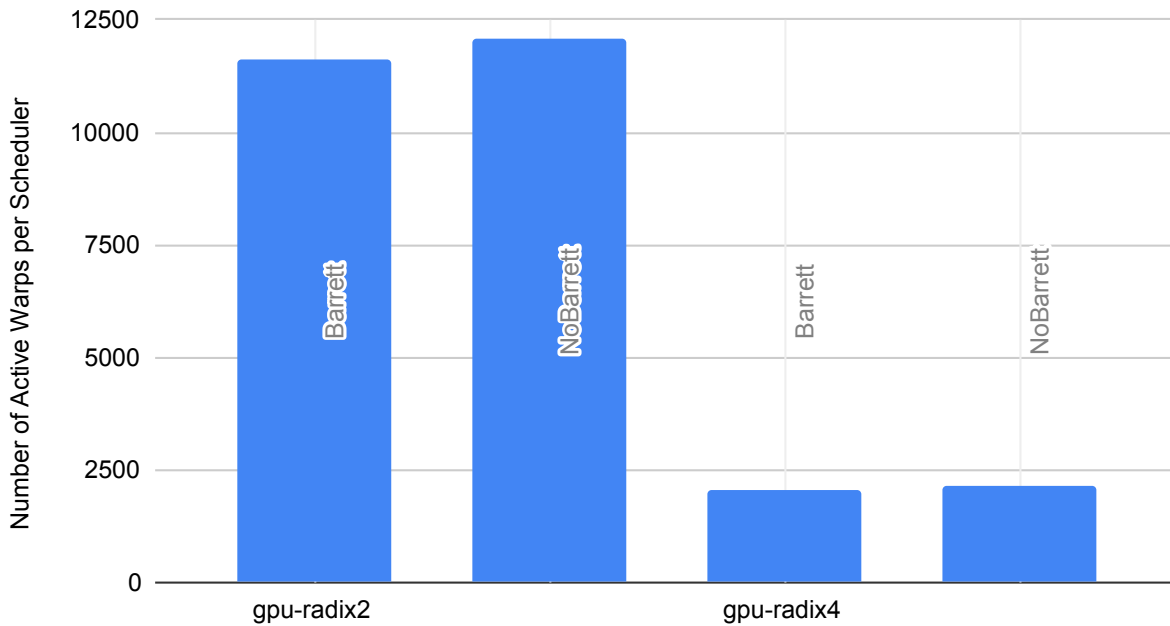
DRAM Throughput GPU radix 2 vs radix 4



Enabling Barrett reduction increases DRAM throughput across both radix implementations. Under NoBarrett, while the SMs are stalled computing these slow divisions, the memory subsystem remains idle. The Barrett reduction causes the threads complete their computation much faster, they issue their global memory requests much sooner, pulling data from VRAM at a higher frequency and increasing the overall DRAM bandwidth utilization. Additionally, since Barrett implementation also executes faster, the DRAM throughput increases due to the reduced execution time.

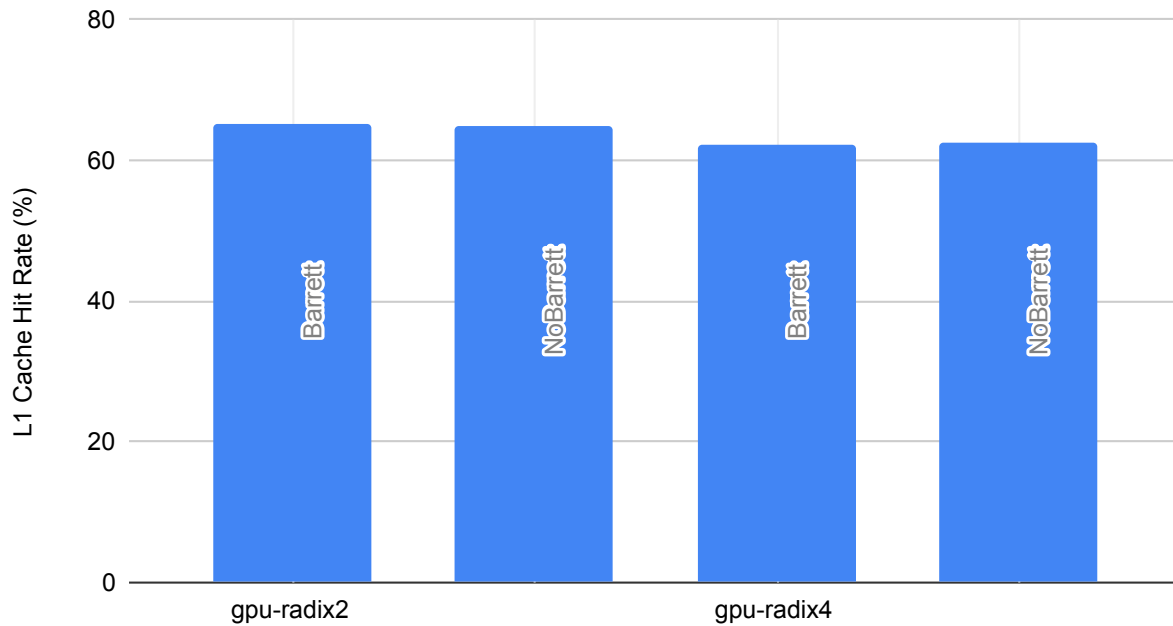
Furthermore, the radix 4 kernels consistently achieve lower DRAM throughput than their radix 2 counterparts. This behavior is due to the higher arithmetic intensity of the radix 4 NTT, which effectively merges two stages of a radix 2 NTT into a single pass. Although a radix 4 thread processes more inputs simultaneously, it halves the total number of kernel launches and global memory round-trips required for the complete algorithm, resulting in lower overall traffic to the DRAM subsystem per unit of work. Considering the reduced hardware occupancy discussed previously, the radix 4 kernel limits the number of active threads available to generate concurrent memory requests, preventing it from saturating the VRAM bandwidth to the same degree as the radix 2 kernel.

Active Warps per Scheduler GPU radix 2 vs radix 4



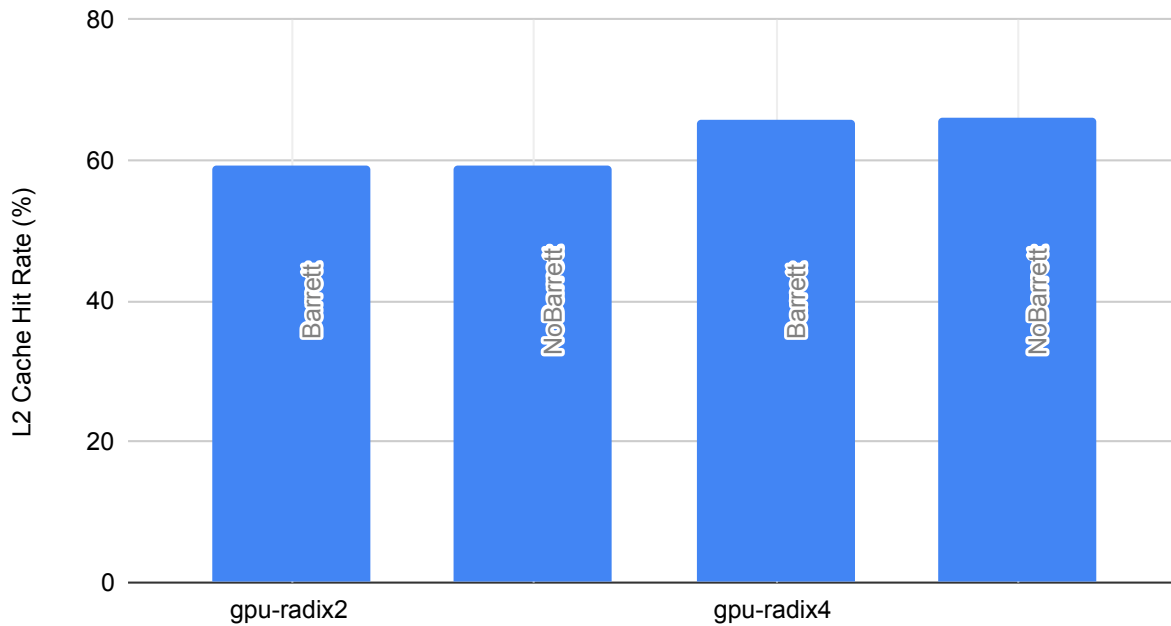
As depicted in the figure above, the sum of active warps per scheduler exhibits a drastic reduction when moving from the radix 2 to the radix 4 implementation, dropping from approximately 12,000 to just over 2,000. This massive decrease is the result of two compounding factors established in previous metrics. First, the high register pressure inherent to the radix 4 kernel lowers the instantaneous hardware occupancy, restricting the maximum number of warps a Streaming Multiprocessor can concurrently manage. Second, because radix 4 processes four coefficients per thread and effectively merges two traditional radix 2 stages into a single pass, it demands substantially fewer total threads and kernel launches to complete the transform. Furthermore, within each radix configuration, enabling Barrett reduction causes a slight dip in the total active warps. This aligns with the previously observed reduction in hardware occupancy, as the Barrett algorithm allocates additional registers for precomputed values such as the μ factor, which slightly tightens the hardware limits on warp scheduling.

L1 Cache Hit Rate GPU radix 2 vs radix 4



As observed in the figure above, the L1 cache hit rates show minimal variance between the Barrett and NoBarrett implementations. This is expected, as enabling Barrett reduction modifies only the arithmetic instructions executed by the ALUs and does not alter the underlying memory access patterns or the data payload. Comparing the radix architectures, radix 2 achieves slightly higher L1 hit rates (approximately 65%) compared to radix 4 (approximately 62%). The overall relatively moderate hit rates for both implementations are characteristic of the Number Theoretic Transform, which relies heavily on power-of-two strided memory accesses that naturally lead to cache thrashing in L1 at larger stage lengths. The minor advantage for radix 2 likely stems from its smaller working set per thread, managing fewer inputs and twiddle factors simultaneously compared to the higher register pressure of radix 4.

L2 Cache Hit Rate GPU radix 2 vs radix 4



Conversely, the L2 cache hit rate demonstrates a distinct advantage for the radix 4 implementation, achieving approximately 66% compared to the 59% observed in radix 2. Similar to the L1 cache results, the choice of modular reduction algorithm (Barrett versus NoBarrett) has negligible impact on the L2 hit rate. The superior L2 cache performance for radix 4 is a direct consequence of its higher arithmetic intensity. By computing the mathematical equivalent of two radix 2 stages in a single kernel execution, the radix 4 algorithm effectively reuses the data loaded into the larger, shared L2 cache before it is evicted back to DRAM. This significant reduction in total kernel launches and global memory round-trips over the entirety of the application allows radix 4 to capitalize on L2 temporal locality much more efficiently than the radix 2 approach.

5. Future Work

Multi-Dimensional Transforms

Because the standard 1-Dimensional NTT is limited by strided memory access patterns that reduce cache hit rates, alternative data layouts can be used to achieve better performance. The 2D NTT provides such an alternative by mapping the linear 1D polynomial array into a 2D matrix where the NTT is computed first across the rows of the matrix, and then across the columns. This approach ensures that memory accesses remain physically localized in continuous memory blocks, which increases sequential cache hits and improves memory coalescing.